Research AI: Generative AI-Based Research Paper Intelligence System

A Project report submitted to the

DEPARTMENT OF COMPUTER APPLICATIONS

in partial fulfilment of the requirements

for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS (HONOURS) IN DS

By

SREE NITHI G

RA2331242010043

Under the Guidance of

Dr. BRINDHA.S, M.Sc., M.Phil., Ph.D.



SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

Faculty of Science and Humanities

Kattankulathur– 603306

October 2027

BONAFIDE CERTIFICATE

This is to certify that the project report titled "**Research AI: Generative AI-Based Research Paper Intelligence System**" is a Bonafide work carried out by **SREE NITHI G (RA2331242010043)** Under my supervision for the award of the Degree of Bachelor of Computer Applications (Honours) in Data Science. To my knowledge the work reported herein is the original work done by this student.

Dr. BRINDHA.S
Assistant Professor,
Department of Computer Applications

Dr. P.J ARUL LEENA ROSE
Professor& Head,
Department of Computer Applications

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

With profound gratitude to the Almighty, I take this chance to thank people who helped me to complete this project.

I take this as a right opportunity to say THANKS to my parents who are there to stand with me always with the words “YOU CAN”.

I wish to express my sincere gratitude to **Dr. T.R. Paarivendhar**, Chancellor, SRM Institute of Science and Technology who gave us the platform to establish me to reach greater heights.

I am thankful to **Dr. A. Duraisamy**, Dean, Faculty of Science and Humanities, SRM Institute of Science & Technology for their unwavering support throughout my project.

I earnestly thank **Dr. S. Albert Antony Raj**, Professor and Deputy Dean, College of Sciences, Faculty of science and Humanities, who always encourage me to do novel things.

I earnestly thanks to **Dr. P. J. Arul Leena Rose**, Professor and Head, Department of Computer Applications, Faculty of Science and Humanities, for her valuable guidance and support to execute all incline in learning.

It is my delight to thank my project guide **Dr. Brindha S**, Assistant Professor, Department of computer Applications, for her help, support, encouragement, suggestions, and guidance throughout the development phases of the project.

I convey my gratitude to all the faculty members of the department who extended their support through valuable comments and suggestions during the reviews.

A great note of gratitude to friends and people who are known and unknown to me who helped in carrying out this project work a successful one.

SREE NITHI G

TABLE OF CONTENTS

Chapter & Title	Sections & Key Coverage	Page
1. INTRODUCTION	1.1 Background, 1.2 Problem Statement, 1.3 Objectives, 1.4 Scope, 1.5 Organisation	**6**
2. LITERATURE STUDY	2.1 Introduction, 2.2 Relevant Research, 2.3 Existing Approaches, 2.4 Gap, 2.5 Summary	**9**
3. SOFTWARE REQUIREMENT ANALYSIS	3.1 Hardware Specification, 3.2 Software Specification, 3.3 Features	**13**
4. SYSTEM ANALYSIS	4.1 Requirements (Functional & Non-Functional), 4.2 Limitations, 4.3 Proposed System, 4.4 SRS	**15**
5. SYSTEM DESIGN	5.1 Architecture, 5.2 Workflow, 5.3 Use Case, 5.4 Logical Model, 5.5 Activity, 5.6 DFD	**18**
6. SYSTEM IMPLEMENTATION	6.1 Ingestion & Storage, 6.2 Module Breakdown, 6.3 Validation Checks, 6.4 Summary	**26**
7. TESTING	7.1 Test Cases (TC-01..21), 7.2 Unit Testing, 7.3 Integrated Testing & Audit Results	**29**
8. RESULTS AND CONCLUSION	8.1 Results, 8.2 Limitations (Synthetic Provenance), 8.3 Future Enhancements, 8.4 Conclusion	**33**
9. APPENDICES	9.1 Startup Instructions, 9.2 API Examples, 9.3 Source Code Excerpts, 9.4 User Guide, 9.5 Glossary	**35**
10. BIBLIOGRAPHY	10.1 Books & Research Publications, 10.2 Official Technical Documentation	**39**

ABSTRACT

PARCS: Performance Anomaly & Regression Correlation System

Modern laptop computers equipped with Microsoft Windows Modern Standby (S0 Low Power Idle, or S0ix) are architected to maintain continuous network connectivity and rapid instant-on wake performance while conserving battery power during system sleep states. To achieve competitive battery longevity, the operating system and platform hardware must achieve deep hardware low-power states, formally designated as Deepest Runtime Idle Power State (DRIPS), targeting an idle residency exceeding 95% of sleep duration. However, in practice, Modern Standby performance frequently degrades due to misbehaving device drivers, faulty power management policies, operating system hotfixes, or uncoordinated background services that prevent the SoC from entering or sustaining DRIPS states. The native diagnostic utility provided by Windows, powercfg /sleepstudy, operates strictly in a stateless, session-by-session manner, lacking longitudinal persistence, machine-specific statistical baselines, multi-session trend analysis, and automated correlation with system changes.

This project presents PARCS (Performance Anomaly & Regression Correlation System), a data-engineering-first telemetry analytics application designed to continuously ingest, persist, analyze, and diagnose Modern Standby behavior. PARCS establishes a robust, decoupled, and zero-server architecture: Windows telemetry is extracted via modular collectors (SleepStudy, installed driver snapshots, Windows Update installation histories, and Windows Event Log power-state records), preserved in raw JSON format, cleaned and schema-validated via an ETL pipeline, and persisted into an immutable Apache Parquet columnar data lake. Fast, SQL-based analytical queries are executed directly against the columnar store using embedded DuckDB without requiring database servers.

To reliably detect genuine performance degradations without conflating them with normal operational variances, PARCS implements machine-specific statistical baselines utilizing robust non-parametric estimators (Median, MAD, IQR). Individual sessions are scored for point anomalies using Modified Z-scores. To eliminate false alarms triggered by transient battery spikes, PARCS introduces a directional sustained regression detector that requires consecutive qualifying degraded sessions in strict timestamp order. When a sustained regression is established, an explainable temporal change correlation engine searches within configurable pre- and post-regression time windows to identify temporally associated system events (device driver updates, Windows patches, kernel power errors), strictly noting that temporal association does not assert causation.

The analytical capabilities are exposed through a high-performance FastAPI REST backend and visualized in an interactive Streamlit dashboard. In this Minimum Viable Product (MVP) release, the system was exhaustively verified against an automated test suite of 253 test cases (0 failures, 1 maintenance warning), achieving 100% pass rates across isolation, evaluation-scope partitioning (subsequent vs all), zero-MAD fallback handling, insufficient-history readiness guards, and machine-specific synthetic-data provenance. Deterministic synthetic fixtures (22 session records and 4 change records across DEMO-MACHINE-READY and DEMO-MACHINE-SPARSE) demonstrate end-to-end functionality. The report clearly distinguishes synthetic demonstration data from genuine laptop telemetry and details future directions for background daemon execution and enterprise fleet diagnostics.

1. INTRODUCTION

1.1 Background of the Project

Personal computing architectures have undergone a profound paradigm shift over the past decade. Historically, laptop power management relied on the Advanced Configuration and Power Interface (ACPI) S3 state, commonly known as Suspend-to-RAM. In an S3 sleep transition, the central processing unit (CPU), system clocks, and peripheral buses are powered down entirely, leaving only system memory actively refreshed. While S3 sleep yielded predictable, near-zero battery consumption, it suffered from notable operational disadvantages: waking the machine required several seconds of hardware re-initialization, and the system remained completely inert, incapable of receiving background updates, push notifications, or network maintenance.

To deliver a modern smartphone-like user experience—characterized by instant-on resumption (under 500 milliseconds) and continuous background synchronization—Microsoft and silicon vendors (Intel, AMD, Qualcomm) introduced Modern Standby, formally designated as S0 Low Power Idle (S0ix). Unlike legacy S3 sleep, a machine entering Modern Standby remains in the ACPI S0 working state. The operating system places the processor cores into deep C-states (such as PC10), switches off display pipelines, suspends non-critical user-mode processes, and signals the platform hardware to enter the Deepest Runtime Idle Power State (DRIPS). When functioning correctly, the platform achieves DRIPS residency exceeding 95% of the sleep duration, achieving milliwatt-level quiescent power consumption.

However, Modern Standby places tremendous reliance on the holistic cooperation of the hardware and software ecosystem. Because the platform remains awake in S0, any misconfigured peripheral device driver, uncoordinated network wake packet, background audio stream, or corrupt system service can prevent the hardware power controller from entering DRIPS. When this occurs, laptops experience severe battery drain while sleeping in backpacks, excessive thermal dissipation, fan spinning during transport, and unexpected battery depletion upon opening the lid. Diagnosing these failures represents one of the most persistent and frustrating challenges in modern PC systems engineering.

1.2 Problem Statement

Despite the pervasive nature of Modern Standby battery drain issues, existing operating system diagnostics suffer from severe architectural and operational limitations:

- **Stateless Diagnostic Tooling:** The primary built-in diagnostic tool, powercfg /sleepstudy, produces isolated, single-session HTML or XML reports. It lacks longitudinal memory, meaning it cannot track how a machine's sleep efficiency evolves across days, weeks, or months.
- **Absence of Machine-Specific Baselines:** Standard battery drain rates vary significantly between different laptop models depending on battery capacity, installed memory, and attached peripherals. Static thresholding produces unacceptable rates of false alarms on high-performance machines and missed detections on ultra-low-power systems.
- **Conflation of Transient Spikes with Sustained Regressions:** Transient factors—such as an active user-initiated download, temporary background virus scan, or brief audio playback—can cause an isolated session to experience poor DRIPS residency. Existing tools treat every isolated spike as an anomaly, failing to distinguish temporary operational blips from genuine, persistent regressions.
- **Disconnected System Event Telemetry:** When sleep degradation occurs, diagnosing the underlying cause requires manually cross-referencing SleepStudy data with device manager snapshots, Windows Update history, and Windows Event Viewer logs. These telemetry sources are siloed in disparate formats, making root-cause triage time-consuming, tedious, and error-prone.

- **Lack of Lightweight, Privacy-Preserving Local Analytics:** Many commercial endpoint management tools upload extensive proprietary telemetry to cloud backends, introducing data privacy concerns, latency, subscription costs, and heavyweight database infrastructure dependencies.

1.3 Objectives of the Project

The overarching objective of the PARCS project is to develop an automated, explainable, and zero-server data engineering telemetry system that tracks Windows Modern Standby performance over time, detects statistically grounded regressions, and correlates performance changes with recorded system events. The specific technical objectives are as follows:

- **Objective 1 (Telemetry Collection):** To engineer modular, Windows-native telemetry collectors for SleepStudy reports, installed driver snapshots, Windows Update installation history, and Windows Event Log power transitions.
- **Objective 2 (Data Engineering & Normalization):** To implement an immutable raw-data preservation layer and an ETL pipeline that validates and normalizes diverse telemetry sources into standardized tabular data structures.
- **Objective 3 (Columnar Persistence & Analytical Store):** To persist normalized telemetry in Apache Parquet columnar files and enable sub-second SQL analytical querying via embedded DuckDB without running an external database daemon.
- **Objective 4 (Machine-Specific Baselines):** To compute machine-specific, robust statistical baselines (Median, MAD, IQR) strictly partitioned by machine identity, enforcing observation readiness requirements (minimum 10 observations) and zero-MAD fallbacks.
- **Objective 5 (Anomaly & Sustained Regression Detection):** To detect explainable point anomalies using robust Modified Z-scores and identify sustained performance regressions via directional consecutive degradation streaks (minimum 3 sessions) in chronological order.
- **Objective 6 (Temporal Change Correlation):** To correlate identified regressions with recorded system events occurring within configurable temporal windows ([-24 hours, +2 hours]), explicitly preserving non-causal epistemological boundaries.
- **Objective 7 (Service & Interactive Visualization):** To expose analytics via a modular FastAPI REST service and deliver an interactive visual dashboard in Streamlit.
- **Objective 8 (Rigorous Verification & Quality Assurance):** To validate the system through exhaustive unit and integration testing (253 automated tests), verifying prospective baseline isolation, evaluation-scope controls, and synthetic data provenance.

1.4 Scope of the Project

The scope of PARCS is focused specifically on client-side Windows Modern Standby performance telemetry and longitudinal diagnostics. The functional boundaries of the current implementation encompass:

- **Supported Telemetry:** Automated extraction from powercfg /sleepstudy, Windows Plug-and-Play (PnP) driver enumeration, Windows Update hotfix metadata, and Microsoft-Windows-Kernel-Power event logs.
- **Analytical Data Architecture:** Standardized Parquet datasets (data/processed/sleepstudy.parquet and data/processed/changes.parquet) querying in-process through DuckDB.
- **Core Performance Metrics:** Evaluation of DRIPS percentage, battery drain rate (mW), and total battery drain percentage against machine-specific baselines.
- **Prospective Partitioning:** Evaluation-scope controls (subsequent prospective evaluation vs all-scope complete dataset scoring) with strict contamination guards.

- **Demonstration Scope:** Deterministic synthetic fixtures (DEMO-MACHINE-READY with 14 baseline + 4 degraded sessions, and DEMO-MACHINE-SPARSE with 4 sessions), explicitly tagged with is_synthetic: True.

Explicit Non-Goals and Out-of-Scope Items: PARCS does NOT modify kernel-mode ACPI drivers, alter firmware power registers, execute automated driver rollback or uninstall actions, or ingest proprietary cloud telemetry. Genuine physical laptop telemetry collection is implemented in collectors but was not deployed long-term; all reported end-to-end multi-session audit results are based strictly on synthetic demonstration fixtures.

1.5 Organisation of the Report

This project report is organized into ten comprehensive chapters designed to document the development, architecture, and empirical verification of PARCS in a structured, academic manner:

- **Chapter 1 (Introduction):** Introduces the operational background of Windows Modern Standby, articulates the problem statement, defines project objectives, and outlines the functional scope.
- **Chapter 2 (Literature Study):** Surveys foundational literature in ACPI power management, robust statistical process monitoring, time-series anomaly detection, temporal event correlation, and columnar analytics, identifying the technical research gap.
- **Chapter 3 (Software Requirement Analysis):** Details hardware configurations, software technology stacks (Python, Parquet, DuckDB, FastAPI, Streamlit, pytest), and technical feature rationale.
- **Chapter 4 (System Analysis):** Presents the formal Software Requirement Specification (SRS), enumerating functional and non-functional requirements and analyzing architectural constraints.
- **Chapter 5 (System Design):** Details the multi-tiered system design, including the system architecture, software workflow, use case model, logical component structure, activity flow, and Level-0/Level-1 data flow diagrams.
- **Chapter 6 (System Implementation):** Documents the concrete implementation of collectors, ETL pipelines, Parquet/DuckDB storage, baseline estimators, streak-based regression detection, correlation algorithms, API routes, and dashboard interfaces.
- **Chapter 7 (Testing):** Presents the testing methodology, formal test case specifications, unit testing regimes, and integrated acceptance audit results (253 passing automated tests, 0 failures, 1 warning).
- **Chapter 8 (Results and Conclusion):** Synthesizes experimental results, articulates system limitations, defines future research directions, and presents concluding observations.
- **Chapter 9 (Appendices):** Supplies supplementary technical materials, including startup instructions, API payloads, representative source code excerpts, user documentation, and an academic glossary.
- **Chapter 10 (Bibliography):** Lists authentic academic books, research papers, and official technical documentation consulted during project execution.

2. LITERATURE STUDY

2.1 Introduction

A rigorous literature study establishes the theoretical foundations, algorithmic paradigms, and architectural patterns relevant to the problem of diagnosing Windows Modern Standby power regressions. In systems engineering, performance degradation during quiescent sleep states cannot be addressed purely through intuitive heuristic checks; it requires understanding low-power hardware states, statistical process control, robust anomaly detection in noisy telemetry, time-series regression isolation, and non-causal event correlation. Furthermore, storing and querying longitudinal endpoint telemetry necessitates an analysis of modern columnar analytical file formats and embedded in-process SQL engines. This chapter surveys peer-reviewed literature and authoritative technical frameworks across these domains, analyzes existing diagnostic approaches, and identifies the core research gap that PARCS addresses.

2.2 Review of Relevant Research and Technical Concepts

2.2.1 ACPI Power States and Windows Modern Standby Architecture

The Advanced Configuration and Power Interface (ACPI) Specification (ACPI Specification Working Group, 2022) defines the global system power states (G-states), sleep states (S-states), and device power states (D-states). Historically, laptops relied on ACPI S3 (Suspend-to-RAM) to achieve quiescent low power. In S3, the CPU and system clocks are disabled, and volatile memory context is refreshed in self-refresh mode. Modern computing platforms, however, increasingly adopt ACPI S0 Low Power Idle (S0ix), branded on Microsoft Windows as Modern Standby or Connected Standby (Microsoft Hardware Developer Center, 2023).

In Modern Standby, the platform remains within ACPI S0. Low power is achieved opportunistically when the operating system directs all platform components and peripherals into low-power device states (D3hot/D3cold), permitting the SoC power management controller to transition into Deepest Runtime Idle Power State (DRIPS). Popp et al. (2018) analyzed SoC power state transitions in mobile architectures, observing that achieving target battery longevity requires sustained DRIPS residency of at least 95% of total sleep time. When a single platform component—such as a PCIe Wi-Fi transceiver, audio codec, or USB host controller—fails to enter D3, the power management controller is blocked from initiating hardware DRIPS (SW_DRIPS vs HW_DRIPS mismatch). Consequently, the CPU remains trapped in shallow C-states (C2 or C3), increasing quiescent power consumption by a factor of 10 to 50 times baseline expectations.

2.2.2 Robust Statistical Process Monitoring and Dispersion Estimators

Statistical process monitoring in systems telemetry requires estimating typical baseline behavior in the presence of noisy, skewed, or contaminated observations. Classical parametric statistics rely on sample mean (mu) and standard deviation (sigma). However, as established in foundational robust statistics literature (Hampel, 1974; Rousseeuw & Croux, 1993), mean and standard deviation possess a breakdown point of 0%, meaning that a single extreme outlier (such as an overnight Windows Update maintenance session) can arbitrarily distort the baseline mean and inflate the standard deviation, masking subsequent degradations (the masking effect) or erroneously flagging normal variations (the swamping effect).

To achieve robustness against outliers and distributional skewness, modern telemetry frameworks employ estimators with high breakdown points (up to 50%). The median represents the optimal robust

location estimator, while the Median Absolute Deviation (MAD) serves as the primary dispersion estimator:

Formulation: MAD = median(|x_i - median(X)|)

Under a standard normal distribution, MAD scaled by 1.4826 asymptotically estimates standard deviation. Iglewicz and Hoaglin (1993) introduced the Modified Z-score (M_i) for outlier detection:

Modified Z-score: M_i = 0.6745 * |x_i - median(X)| / MAD

In practical systems engineering, when identical observations occur repeatedly (such as laptops consistently achieving exactly 99% DRIPS residency), MAD collapses to zero. A robust baseline implementation must incorporate documented fallback mechanisms, utilizing the Interquartile Range (IQR / 1.3490) or a minimal positive epsilon, to prevent division-by-zero singularities while preserving scale sensitivity.

2.2.3 Time-Series Anomaly and Sustained Regression Detection

A critical distinction in systems performance analysis is the difference between point anomalies and sustained regressions. A point anomaly represents an isolated, single-session deviation from baseline expectation—such as an active user downloading a game update while the laptop lid is closed. In contrast, a sustained regression represents a permanent or persistent shift in platform behavior resulting from a structural change in the system (e.g. an incompatible device driver update).

Basseville and Nikiforov (1993) formalized the detection of abrupt and persistent changes in dynamic time series. In industrial software reliability, detecting persistent shifts without incurring unacceptable false-positive rates requires streak-based sequential evaluation: an alert is raised only when a minimum run length (k consecutive sessions) consistently crosses a directional harmful deviation threshold in chronological order. Furthermore, prospective evaluation requires that degraded observations must not contaminate the historical reference window used to evaluate them, a principle known as reference partition isolation.

2.2.4 Temporal Event Correlation and Non-Causal Systems Diagnostics

When a performance regression is detected, engineers must identify the likely environmental factors associated with the degradation. Oliner et al. (2012) conducted a comprehensive survey of anomalies and event correlation in software systems, emphasizing that temporal proximity between system events (driver updates, OS hotfixes, service configuration changes) and regression onset provides the primary investigative clue for root-cause triage.

However, Pearl (2009) established in foundational causal inference literature that temporal association does not equal causality. An event occurring before a regression (e.g. a routine audio driver update) may be purely coincidental, while the true unrecorded root cause is an environmental temperature change or background malware. Diagnostic software must therefore maintain strict epistemological humility: systems must explicitly classify relationships as temporal correlations within defined time windows ([-W_before, +W_after]), providing evidence for human investigation rather than asserting unsubstantiated causal claims.

2.2.5 Modern Columnar Analytical Storage and In-Process Query Engines

Telemetry ingestion architectures have evolved away from traditional row-oriented relational databases (RDBMS) toward columnar storage formats. Abadi et al. (2008) demonstrated that columnar architectures achieve order-of-magnitude performance improvements for analytical queries involving aggregate computations over selected metric columns. Apache Parquet (Apache Software Foundation,

2023) has emerged as the open industry standard for columnar persistence, featuring run-length encoding, dictionary compression, and metadata-driven chunk skipping.

Concurrently, Raasveldt and Mühleisen (2019) introduced DuckDB, an embedded, vectorized analytical database management system. Unlike SQLite (which is row-oriented and optimized for transactional OLTP workloads) and client-server databases like PostgreSQL or ClickHouse (which require external daemon management and network socket communication), DuckDB operates directly inside the host application process. DuckDB executes zero-copy SQL queries against Parquet files directly on the local filesystem, achieving microsecond query latencies with zero external infrastructure overhead.

2.3 Analysis of Existing Approaches

To evaluate the technical positioning of PARCS, existing commercial and operating system diagnostic tools were evaluated across key functional capabilities: longitudinal persistence, machine-specific baselines, sustained regression detection, event correlation, local-first execution, and zero-server infrastructure.

Diagnostic Tool	Storage Architecture	Baseline Strategy	Regression Model	Event Correlation	Zero Server
powercfg /sleepstudy	Stateless HTML/XML	None (Static threshold)	None (Single-session)	None (Isolated)	Yes
Windows Perf Analyzer	ETW trace files (.etl)	None (Manual inspection)	None (Manual timeline)	Manual ETW cross-check	Yes
Reliability Monitor	Local WMI/Registry	None (Daily score index)	None (Crash frequency)	Visual event overlay	Yes
Datadog / Dynatrace	Cloud SaaS Data Lake	Parametric (Mean/Std)	Statistical / ML anomaly	Automated APM traces	No (Heavy Cloud)
PARCS (Proposed)	Parquet + DuckDB	Robust (Median, MAD, IQR)	Streak-based consecutive	Temporal window correlation	Yes (Local First)

As summarized in the comparison table, native operating system tools provide deep hardware inspection but lack longitudinal memory and automated correlation. Enterprise APM suites offer longitudinal tracking but require heavyweight cloud infrastructure, subscription licensing, and proprietary data offloading. PARCS uniquely bridges this divide by delivering advanced analytical capabilities in a lightweight, local-first architecture.

2.4 Research Gap

The survey of existing literature and diagnostic tooling reveals a prominent technical gap: there is an acute lack of an automated, privacy-preserving, zero-infrastructure data engineering pipeline that can continuously monitor Windows Modern Standby telemetry, construct robust machine-isolated statistical baselines, detect sustained performance regressions, and temporally correlate degradations with

recorded system changes without relying on external database daemons or cloud services. PARCS is explicitly engineered to address this gap.

2.5 Summary

This chapter reviewed foundational concepts in ACPI S0ix Modern Standby, robust statistics (Median, MAD, IQR), Modified Z-scores, change point detection, non-causal event correlation, and columnar analytics with Parquet and DuckDB. These technical foundations establish the theoretical framework implemented in PARCS, detailed in subsequent chapters.

3. SOFTWARE REQUIREMENT ANALYSIS

3.1 Hardware Specification

Hardware requirements define the minimum and recommended physical computing resources necessary to execute the PARCS telemetry collection, ETL pipeline, DuckDB analytical engine, FastAPI REST backend, and Streamlit user interface. Because PARCS operates entirely in-process using embedded columnar storage, its hardware footprint is exceptionally lightweight.

Hardware Component	Minimum Requirement	Recommended Specification	Evaluation Platform Role
Processor (CPU)	x86-64 Dual-Core (1.8 GHz)	Intel Core i5 / AMD Ryzen 5 Quad-Core+	Host telemetry generation & DuckDB execution
System Memory (RAM)	4 GB DDR4	8 GB to 16 GB DDR4/DDR5	In-memory pandas/DuckDB vectorized execution
Storage Space	200 MB free disk space	1 GB NVMe SSD space	Raw JSON reports & processed Parquet datasets
Display Resolution	1280 x 720 (HD)	1920 x 1080 (Full HD) or higher	Streamlit responsive dashboard rendering
Power Management	ACPI S0 Low Power Idle support	Windows Modern Standby certified hardware	SleepStudy telemetry generation on Windows

3.2 Software Specification

The software specification enumerates the programming languages, runtime environments, core libraries, web frameworks, and testing suites utilized in the implementation of PARCS, confirmed directly from the repository's requirements.txt.

Software Component	Version Specification	Functional Role in PARCS Architecture
Operating System	Windows 10 / 11 (64-bit)	Host environment for powercfg, PnP device enumeration, and event logs
Python Runtime	Python 3.10 to 3.14 (Tested on 3.14.5)	Core programming language for collectors, ETL, analytics, and API
DuckDB	duckdb >= 0.9.0	Embedded in-process SQL analytical query engine over Parquet tables
Apache Arrow / PyArrow	pyarrow >= 14.0.0	Columnar memory layout and high-performance Parquet reading/writing

Software Component	Version Specification	Functional Role in PARCS Architecture
Pandas	pandas >= 2.0.0	Tabular data manipulation, time-series indexing, and feature engineering
FastAPI	fastapi >= 0.100.0	Asynchronous REST API framework exposing analytical endpoints
Uvicorn	uvicorn >= 0.20.0	ASGI web server implementation for FastAPI backend hosting
Streamlit	streamlit >= 1.30.0	Interactive data visualization dashboard and telemetry exploration UI
HTTPX	httpx >= 0.25.0	Asynchronous and synchronous HTTP client for dashboard API communication
Pydantic	pydantic >= 2.0.0	Data validation and settings management using Python type annotations
pytest	pytest >= 7.0.0	Automated test runner executing unit and integration verification suites

3.3 About the Software and Its Features

PARCS is implemented in modern Python, leveraging an optimal data-engineering stack designed for high throughput, low memory consumption, and zero external daemon management:

- **Python Runtime:** Python 3.14 provides a highly optimized runtime with modern type annotation support and robust system automation capabilities, allowing seamless interaction with Windows command-line utilities and system APIs.
- **Apache Parquet:** Apache Parquet delivers efficient columnar compression, schema enforcement, and rapid selective scans. Telemetry data is partitioned immutably, ensuring long-term longitudinal persistence without database table bloat.
- **Embedded DuckDB:** DuckDB provides vectorized analytical SQL execution directly against Parquet files. Because DuckDB is embedded as a Python library, PARCS requires no background database service, socket configuration, or database administrative credentials.
- **FastAPI:** FastAPI provides OpenAPI-compliant, type-safe REST endpoints with automatic Pydantic schema validation. Query parameters such as threshold cutoffs, observation counts, and evaluation scopes are validated automatically, rejecting malformed requests with HTTP 422.
- **Streamlit:** Streamlit provides a dynamic, responsive web interface that connects directly to the FastAPI backend or falls back seamlessly to local in-process analytics if the backend service is offline.

4. SYSTEM ANALYSIS

4.1 Requirement Specification

Requirement specification defines the essential functional capabilities and non-functional quality attributes that govern the design, implementation, and verification of PARCS.

4.1.1 Functional Requirements

The functional requirements define the explicit operations performed by PARCS:

- **FR-01 (SleepStudy Collection):** The system shall execute Windows powercfg /sleepstudy via subprocess, extract structured standby sessions (start/end timestamps, duration, DRIPS %, battery drain %, drain rate mW), and preserve the raw HTML/XML report.
- **FR-02 (Driver Telemetry):** The system shall capture timestamped snapshots of installed Plug-and-Play device drivers, preserving device IDs, driver names, versions, providers, and status.
- **FR-03 (Update Telemetry):** The system shall query Windows Update history, capturing hotfix identifiers (KB numbers), descriptions, and installation timestamps.
- **FR-04 (Event Log Telemetry):** The system shall extract Microsoft-Windows-Kernel-Power events within configurable time windows around sleep sessions.
- **FR-05 (Raw Data Preservation):** The system shall store raw collector outputs in timestamped JSON/HTML files under data/raw/ before performing any transformation.
- **FR-06 (ETL Normalization):** The system shall normalize raw session and change data into standardized schemas, validating types and handling missing fields.
- **FR-07 (Parquet Persistence):** The system shall persist normalized sessions into data/processed/sleepstudy.parquet and system changes into data/processed/changes.parquet.
- **FR-08 (DuckDB Analytical Querying):** The system shall provide an in-process DuckDB manager capable of executing SQL aggregations directly over Parquet files.
- **FR-09 (Machine-Isolated Baselines):** The system shall compute Median, MAD, and IQR baselines strictly partitioned by machine_id, marking readiness as 'ready' if observations >= min_observations (default 10) or 'insufficient_history' otherwise.
- **FR-10 (Point Anomaly Detection):** The system shall compute Modified Z-scores against ready baselines, flagging deviations exceeding the threshold (default 3.5) without asserting causality.
- **FR-11 (Sustained Regression Detection):** The system shall detect consecutive qualifying directional degradations (default 3 sessions) in chronological order, isolating reference observations from subsequent evaluation.
- **FR-12 (Temporal Change Correlation):** The system shall identify system changes occurring within configurable temporal windows around regression onset, annotating temporal relations ('before', 'concurrent', 'after').
- **FR-13 (REST API Services):** The system shall expose analytics via FastAPI endpoints (/health, /api/telemetry/summary, /api/analytics/baselines, /api/analytics/anomalies, /api/analytics/regressions, /api/analytics/correlations).
- **FR-14 (Visual Dashboard):** The system shall deliver a multi-tab Streamlit dashboard providing machine selection, metric overviews, baseline inspections, and regression timelines.

4.1.2 Non-Functional Requirements

The non-functional requirements define the quality, reliability, and security constraints of the application:

- **NFR-01 (Machine Isolation):** Observations from different machines shall never be combined or permitted to contaminate one another's baselines or evaluations.
- **NFR-02 (Prospective Evaluation Purity):** Historical reference observations shall be strictly isolated from subsequent evaluation sets to prevent degraded sessions from shifting the baseline used to score themselves.
- **NFR-03 (Non-Causal Explainability):** All regression and correlation outputs shall state statistical and temporal observations without asserting unverified causal claims.
- **NFR-04 (Zero Infrastructure Overhead):** The system shall operate without requiring external database servers, background services, cloud dependencies, or administrative network ports.
- **NFR-05 (Deterministic Repeatability):** All analytical calculations, baseline sort orders, and streak evaluations shall yield identical, deterministic results across repeated executions.
- **NFR-06 (Data Provenance Transparency):** The system shall explicitly distinguish synthetic demonstration fixtures (is_synthetic: True) from genuine physical system telemetry across all layers.

4.2 Characteristics and Limitations of Existing Approaches

Existing techniques for Windows Modern Standby diagnostics rely on disconnected manual workflows. An engineer noticing high battery drain must open an elevated command prompt, run powercfg /sleepstudy, open the resulting HTML report in a web browser, and manually scroll through dozens of sleep sessions. If a poor session is identified, the engineer must manually open Device Manager to check recently updated drivers, open Windows Update Settings to inspect recent hotfixes, and open Windows Event Viewer to inspect Kernel-Power Event ID 506 or 507 entries. This manual triage process is tedious, highly subjective, and incapable of detecting subtle, gradual performance degradation over time.

4.3 Proposed System

PARCS replaces manual, disconnected triage with an automated, data-engineering-first telemetry analytics pipeline. By decoupling collection, normalization, persistence, statistical modeling, and visualization, PARCS establishes an end-to-end automated workflow:

- **Unified Multi-Source Telemetry:** PARCS unifies SleepStudy, driver snapshots, Windows updates, and event logs into a single structured pipeline.
- **Columnar Data Lake:** Using Apache Parquet and DuckDB, PARCS enables instant historical queries without the operational overhead of a relational database server.
- **Machine-Specific Robust Baselines:** Non-parametric Median, MAD, and IQR statistics adapt dynamically to each specific machine's hardware characteristics.
- **Streak-Based Regression Detection:** Requiring consecutive qualifying degradations prevents single-session transient anomalies from triggering false alarms.
- **Explainable Temporal Correlation:** Automated matching of regression onset with recorded changes eliminates manual cross-referencing while preserving non-causal integrity.

4.4 Software Requirement Specification

The Software Requirement Specification (SRS) formalizes the operational environment, input specifications, processing requirements, and expected system outputs for PARCS:

- **System Environment:** Desktop execution environment running Windows 10/11 with Python 3.10+ and required packages installed in a virtual environment (.venv).

- **Input Requirements:** SleepStudy HTML/XML files generated by powercfg, PowerShell driver query JSON structures, Windows Update hotfix lists, and wevtutil event log records.
- **Processing Requirements:** Schema validation, data type normalization, columnar Parquet writing, in-process DuckDB SQL aggregation, robust baseline estimation, streak tracking, and temporal window matching.
- **Output Requirements:** Standardized Parquet tables (sleepstudy.parquet, changes.parquet), JSON REST API responses, and interactive Streamlit UI charts and tables.
- **Constraints and Assumptions:** Collection requires appropriate local Windows permissions; real-machine validation requires physical sleep cycles; synthetic fixtures are utilized for verified automated testing.

5. SYSTEM DESIGN

5.1 Introduction

System design translates the functional and non-functional requirements identified in the system analysis phase into a coherent, robust, and extensible software architecture. In data-intensive systems, design choices dictate not only execution performance and storage efficiency, but also data integrity, modularity, and analytical correctness. PARCS is architected around a strict data-engineering-first philosophy: each stage of the pipeline possesses well-defined input and output contracts, preventing cross-layer coupling and ensuring that analytical assumptions (such as machine isolation and prospective reference purity) are enforced by design. This chapter details the system architecture, software workflow, use case model, logical components, activity flow, data flow diagrams (DFD Level 0 and Level 1), and foundational design considerations.

5.2 System Architecture

The overall system architecture of PARCS is organized into six decoupled tiers spanning telemetry ingestion, raw preservation, ETL normalization, columnar persistence, analytics, and service presentation. Each layer operates strictly within its designated functional boundary:

- **1. Telemetry Ingestion Layer:** Executes Windows-native utilities (powercfg /sleepstudy, Get-PnpDevice, Get-HotFix, wevtutil) to capture hardware, driver, and OS events without performing any business logic or data transformation.
- **2. Raw Storage & ETL Layer:** Immutably archives raw collector output in timestamped JSON/HTML files under data/raw/, and executes schema validation, data cleaning, and canonical type casting.
- **3. Persistence & Query Layer:** Stores normalized telemetry in Apache Parquet columnar files (data/processed/) and provides zero-copy, in-process SQL analytical query capabilities via embedded DuckDB.
- **4. Feature Engineering & Analytics Layer:** Extracts behavioral performance features, constructs machine-isolated robust statistical baselines (Median, MAD, IQR), evaluates Modified Z-score point anomalies, and identifies sustained regressions across consecutive sessions.
- **5. Correlation & Provenance Layer:** Associates detected regressions with recorded system changes within configurable temporal windows, tracking data provenance (is_synthetic flags).
- **6. Service & Presentation Layer:** Exposes analytics via an asynchronous FastAPI REST service and renders responsive charts and diagnostic metrics in a Streamlit web application.

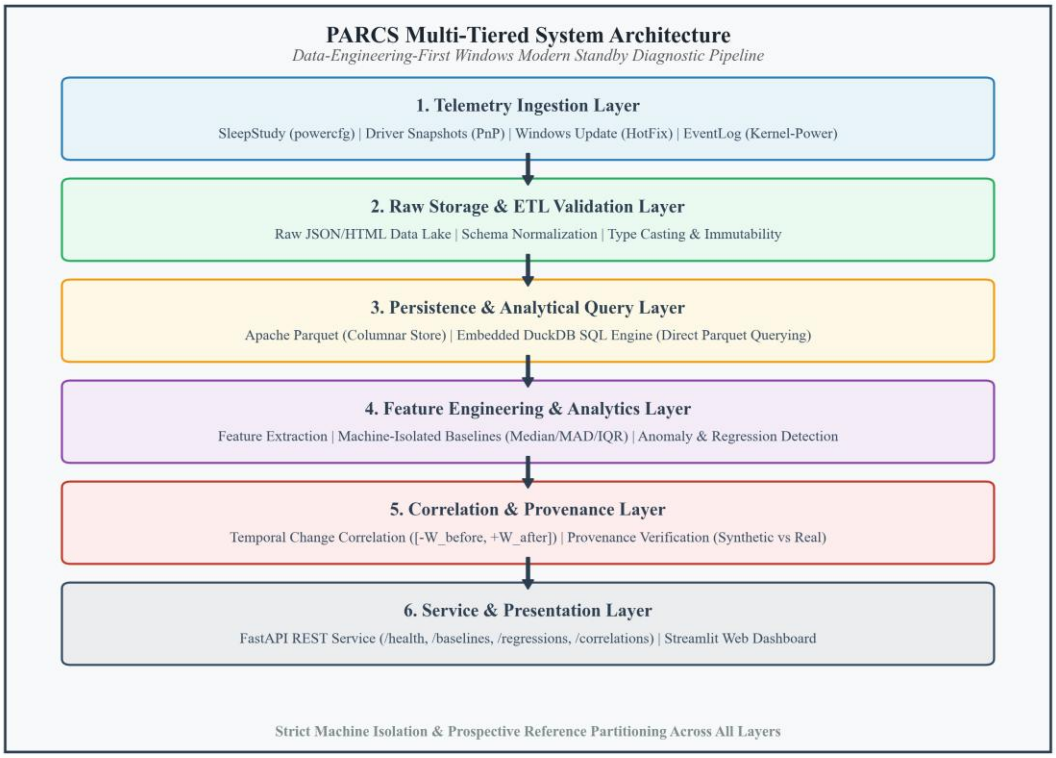


Figure 5.1: Overall System Architecture of PARCS

5.3 Software Workflow Diagram

The software workflow illustrates the chronological data path as raw Windows events are transformed into actionable engineering insights. The workflow enforces unidirectional data flow, ensuring that downstream layers cannot alter or corrupt upstream telemetry.

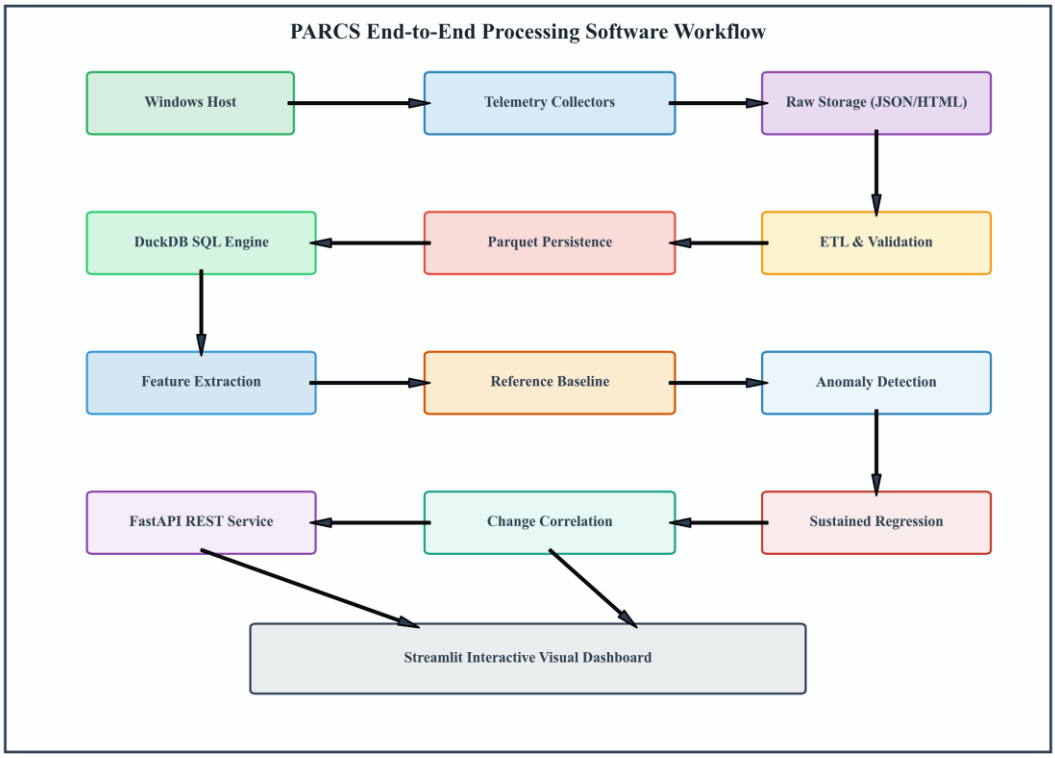


Figure 5.2: End-to-End Processing Software Workflow Diagram

As depicted in Figure 5.2, collection initiates on the Windows host. Structured collector results are preserved in raw storage, validated by the ETL pipeline, and persisted to Parquet. The analytics engine queries Parquet via DuckDB, partitions observations into reference and evaluation sets, builds machine-specific baselines, evaluates anomalies and regressions, and performs non-causal change correlation, serving results via FastAPI and Streamlit.

5.4 Use Case Diagram

The use case diagram models the interactions between the primary human actor—the Performance Engineer or System Administrator—and the core functional boundaries of PARCS.

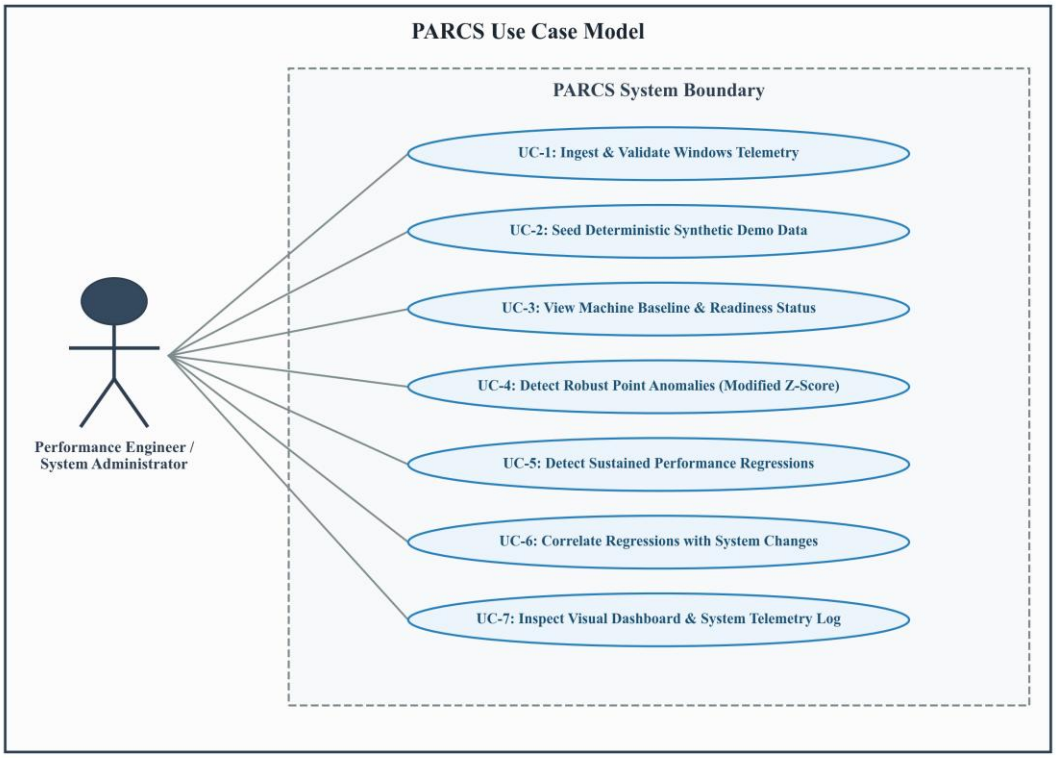


Figure 5.3: Use Case Diagram of PARCS

The primary actor interacts with seven discrete use cases: ingesting telemetry (UC-1), seeding deterministic demo fixtures (UC-2), inspecting machine baselines and readiness statuses (UC-3), detecting point anomalies (UC-4), identifying sustained regressions (UC-5), correlating regressions with system events (UC-6), and exploring telemetry trends through the visual dashboard (UC-7).

5.5 Logical Component and Data Model Design

PARCS is constructed from cohesive Python modules organized into functional subsystems paired with strongly typed columnar data schemas. This modularity ensures separation of concerns between collection, storage, and statistical modeling.

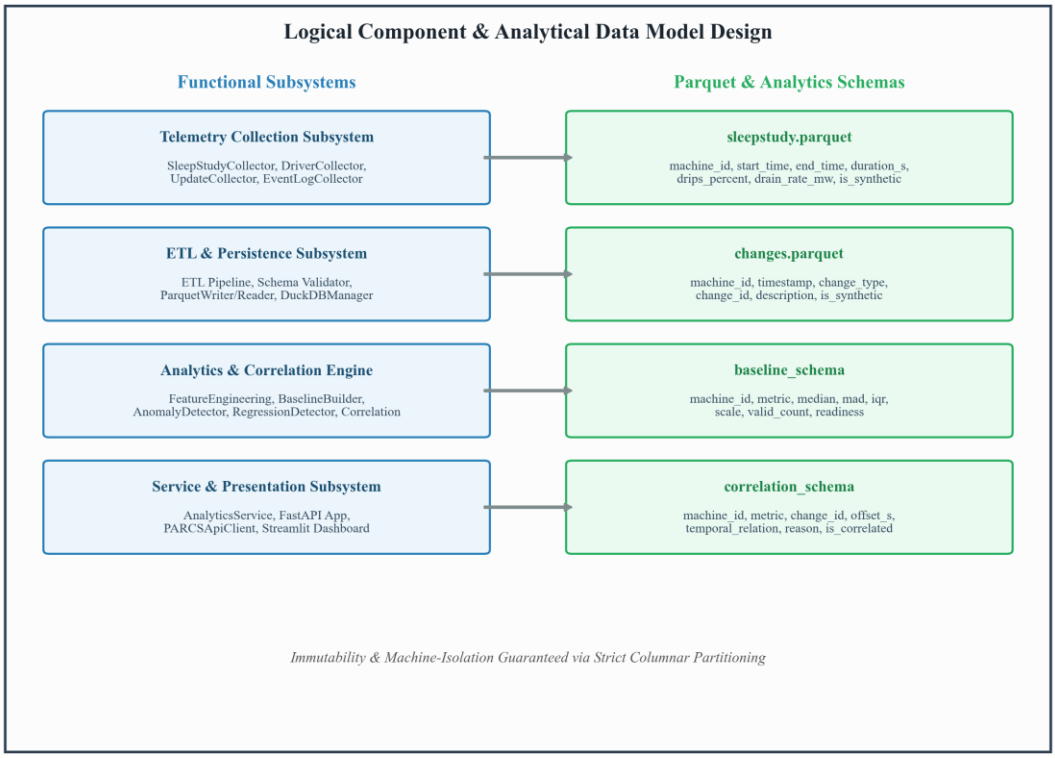


Figure 5.4: Logical Component and Data Model Architecture

The data model design establishes strict columnar contracts across the pipeline. The sleepstudy dataset tracks machine_id, start_time, end_time, duration_seconds, drips_percent, battery_drain_percent, battery_drain_rate_mw, and is_synthetic. The changes dataset records machine_id, timestamp, change_type, change_id, description, and is_synthetic. The baseline schema captures median, mad, iqr, dispersion_scale, valid_count, and readiness, while the correlation schema outputs matched change_id, temporal_relation, offset_seconds, and non-causal explanation strings.

5.6 Activity Diagram

The activity diagram models the dynamic control flow during telemetry analysis, emphasizing decision points around observation count thresholds, baseline readiness, and streak continuity.

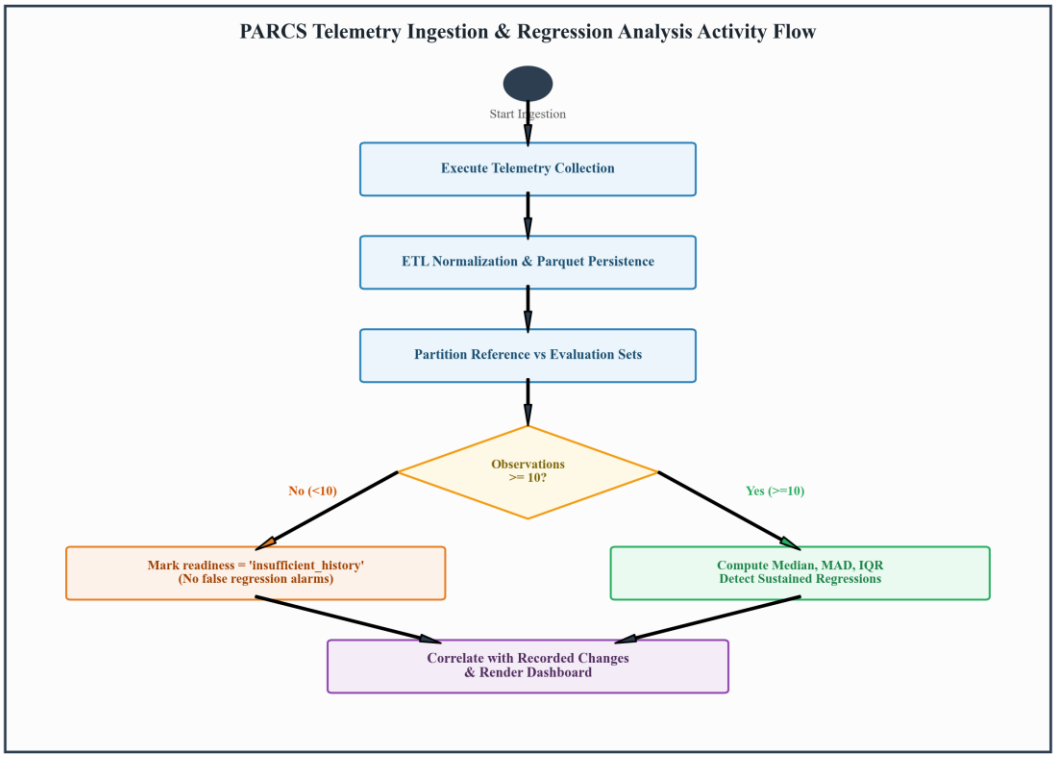


Figure 5.5: Activity Diagram of PARCS Telemetry Ingestion and Analysis

A critical branch in Figure 5.5 occurs during observation count evaluation: if a machine contains fewer than 10 valid historical sessions, its baseline readiness is assigned 'insufficient_history', and the regression detector returns an explicit unready status without raising false alarms. Only machines with ready baselines proceed to streak-based regression evaluation.

5.7 Data Flow Diagram

Data Flow Diagrams (DFDs) model how data moves, transforms, and rests across system boundaries without implying control logic.

5.7.1 Context-Level Data Flow Diagram (DFD Level 0)

The Context-Level DFD treats PARCS as a single unified process interacting with external entities: the Windows Host OS (providing diagnostic telemetry) and the Performance Engineer (submitting analytical queries and receiving visual insights).

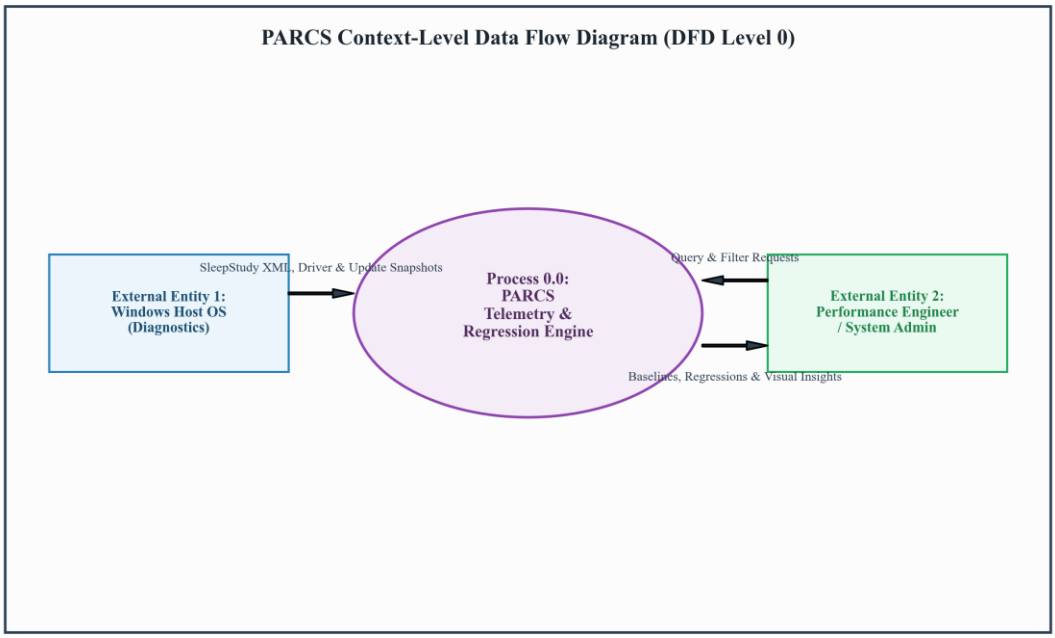


Figure 5.6: Context-Level Data Flow Diagram (DFD Level 0)

5.7.2 Level-1 Data Flow Diagram (DFD Level 1)

The Level-1 DFD decomposes Process 0.0 into seven discrete sub-processes (1.0 Collection, 2.0 ETL, 3.0 Storage/DuckDB Query, 4.0 Baseline Construction, 5.0 Regression Scoring, 6.0 Temporal Correlation, 7.0 Presentation) interacting with three persistent data stores: D1 Raw Storage, D2 Parquet Data Store, and D3 Analytics Cache.

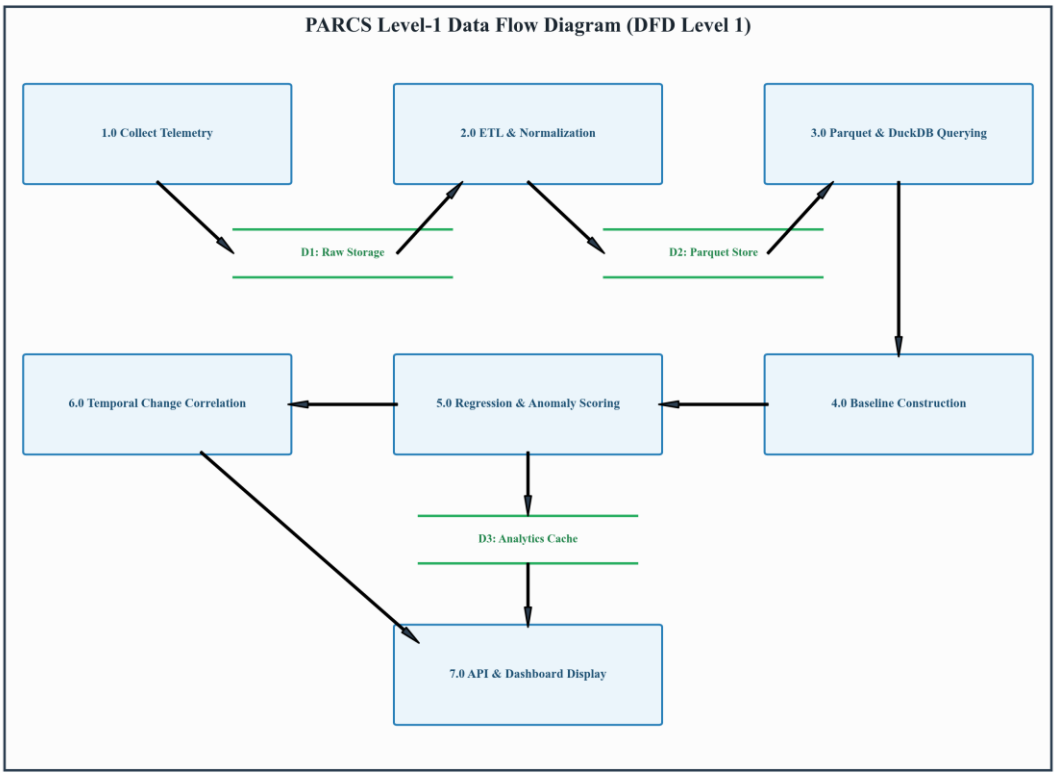


Figure 5.7: Level-1 Data Flow Diagram of PARCS (DFD Level 1)

5.8 Design Considerations

The architecture incorporates four essential engineering design considerations:

- **1. Prospective Reference Partitioning:** To prevent later degraded sessions from contaminating the baseline used to evaluate them, PARCS deterministically splits observations chronologically into reference and subsequent evaluation partitions.
- **2. Strict Machine Isolation:** Observations from different machines are never aggregated. Machine identities are isolated across all pandas groupby and DuckDB query filters.
- **3. Zero-MAD Fallback Handling:** When observations exhibit zero variance (MAD = 0), the system dynamically falls back to IQR / 1.3490 or a minimal epsilon (1e-4) to prevent division-by-zero errors while maintaining sensitivity.
- **4. Non-Causal Epistemological Framing:** All explanations explicitly state temporal offsets (e.g. 'Driver updated 1.5h before regression onset') while declaring that causation is not asserted, preventing engineers from misinterpreting correlation as proof.

5.9 Chapter Summary

This chapter articulated the comprehensive design of PARCS across six architectural layers, complete with formal UML and data flow diagrams. The design enforces machine isolation, prospective reference purity, robust zero-MAD resilience, and explainable correlation, establishing the framework for implementation.

6. SYSTEM IMPLEMENTATION

6.1 Introduction

System implementation translates the design specifications into modular, maintainable, and verifiable Python source code. PARCS is organized under the src/ directory into discrete packages: collectors, etl, storage, analytics, api, dashboard, and utils. This chapter details the technical implementation of each subsystem, documenting the concrete algorithms, contracts, and validation checks implemented in the codebase.

6.2 Module Description

6.2.1 Telemetry Collectors (src/collectors/)

All collectors inherit from the BaseCollector abstract base class defined in src/collectors/base.py, implementing a standardized collect() method that returns a strongly typed CollectorResult namedtuple or dataclass containing collector name, success status, collected_at timestamp, raw_path reference, data payload, and error string. The collection layer comprises four specialized modules:

- **SleepStudyCollector (sleepstudy.py):** Executes powercfg /sleepstudy /output <path> /xml via Python's subprocess.run with check=False and timeout guards. Parses the generated XML/HTML report to extract session start/end timestamps, duration in seconds, DRIPS percentage, total battery drain percentage, and battery drain rate in milliwatts (mW). Preserves the raw report under data/raw/.
- **DriverCollector (drivers.py):** Executes PowerShell Get-PnpDevice to capture a complete snapshot of installed Plug-and-Play device drivers, recording device instance ID, friendly name, driver version, manufacturer/provider, and device status.
- **UpdateCollector (updates.py):** Queries Windows Update history via PowerShell Get-HotFix and Microsoft.Update.Session COM objects, extracting hotfix identifiers (e.g. KB5034204), installation timestamps, and descriptions.
- **EventLogCollector (eventlog.py):** Queries the Windows Event Log via wevtutil or Get-WinEvent, focusing on Microsoft-Windows-Kernel-Power (Event IDs 506 and 507) to capture sleep transitions, wake reasons, and hardware power events around sleep sessions.

6.2.2 Raw Storage Preservation (src/storage/raw.py, paths.py)

To guarantee auditability and protect against data loss during schema migrations, the raw storage module persists all collector payloads into data/raw/ using deterministic timestamped filenames: <collector>-<timestamp>-<hash>.json. Raw files are strictly immutable; once written, they are never updated or truncated.

6.2.3 ETL and Schema Validation (src/etl/)

The ETL pipeline (pipeline.py, sleepstudy.py, transform.py) extracts raw sessions and system changes, applies data cleaning (stripping whitespace, normalizing ISO 8601 UTC timestamps), and validates schemas against canonical column specifications (SLEEPSTUDY_COLUMNS and CHANGE_SCHEMA_COLUMNS). Null and invalid values are handled gracefully, ensuring downstream analytics receive consistent tabular data.

6.2.4 Columnar Parquet Persistence (src/storage/parquet.py)

Normalized datasets are persisted to Apache Parquet format using pyarrow and pandas under data/processed/sleepstudy.parquet and data/processed/changes.parquet. Write operations utilize atomic file swaps to prevent file corruption during concurrent reads.

6.2.5 DuckDB Analytical Storage (src/storage/db.py)

The DuckDBManager establishes an in-process DuckDB connection. It registers the Parquet files as views or queries them directly via DuckDB's read_parquet() function, providing sub-millisecond analytical aggregations without requiring external database server processes.

6.2.6 Feature Engineering (src/analytics/features.py)

The feature engineering module takes normalized SleepStudy sessions and derives analytical metrics, ensuring that start_time and end_time are parsed as timezone-aware UTC timestamps, calculating duration deltas, and validating that drips_percent falls within [0.0, 100.0] and battery_drain_rate_mw is positive.

6.2.7 Statistical Baseline Construction (src/analytics/baseline.py)

The baseline module computes machine-specific non-parametric baselines. For each machine and metric (drips_percent, battery_drain_rate_mw, battery_drain_percent), it calculates the median, MAD, and IQR. If valid_count < min_observations (default 10), readiness is set to 'insufficient_history'. If MAD == 0, it falls back to IQR / 1.3490 or epsilon (1e-4), setting zero_mad_fallback = True.

6.2.8 Point Anomaly Detection (src/analytics/anomaly.py)

The anomaly detector computes Modified Z-scores for each session against the ready baseline. Sessions exceeding the cutoff (threshold = 3.5) are flagged with is_anomaly = True and assigned status = 'anomaly'. If the baseline is unready, status = 'insufficient_history' is returned with is_anomaly = False.

6.2.9 Sustained Regression Detection (src/analytics/regression.py)

The regression detector evaluates observations sequentially in chronological timestamp order. It enforces directional harmful deviations (lower DRIPS %, higher battery drain). A regression event is flagged when at least min_consecutive (default 3) consecutive qualifying sessions cross the threshold. Non-qualifying or missing sessions immediately reset the streak.

6.2.10 Temporal Change Correlation (src/analytics/correlation.py)

When regressions are flagged, the correlation engine searches for system changes on the same machine within the window [start_time - 24 hours, start_time + 2 hours]. It computes the temporal offset in seconds, classifies the relation ('before', 'concurrent', 'after'), and generates an explainable non-causal reason string.

6.2.11 FastAPI Backend Services (src/api/)

The FastAPI backend (app.py, service.py, models.py) provides high-performance asynchronous REST endpoints. AnalyticsService coordinates data loading, prospective partitioning, baseline generation, and regression scoring. All parameters are validated via Pydantic models.

6.2.12 Streamlit Dashboard Application (src/dashboard/)

The dashboard (app.py, client.py) delivers an interactive user interface featuring a sidebar for machine selection and threshold configuration, metric summary cards, and five analytical tabs: Statistical Baselines, Anomaly Detection, Sustained Regressions, Change Correlation, and System Changes Log.

6.3 Validation Checks

PARCS implements six critical validation checks verified by the automated test suite:

- **1. Evaluation-Scope Isolation:** AnalyticsService.partition_machine_features supports 'subsequent' (evaluating only sessions occurring after the reference partition) and 'all' (scoring the entire dataset against the reference). Invalid scope values are rejected with HTTP 422.
- **2. Reference Exclusion:** Reference observations are strictly excluded from subsequent evaluation sets, preventing circular self-scoring.
- **3. Empty Subsequent Handling:** When reference_count covers all available observations, the subsequent evaluation set is empty, and the service returns an empty list ([]) without silently falling back to scoring reference sessions.
- **4. Insufficient History Guard:** Machines with fewer than min_observations (such as DEMO-MACHINE-SPARSE with 4 sessions) receive status 'insufficient_history' and is_regression = False.
- **5. HTTP 422 Validation:** API query parameters (threshold <= 0, min_consecutive < 1, negative windows) are rejected with HTTP 422 Unprocessable Entity.
- **6. Provenance Validation:** Synthetic fixtures are explicitly tagged with is_synthetic: True, and telemetry summaries report machine-specific provenance without false flags.

6.4 Chapter Summary

This chapter detailed the concrete Python implementation of PARCS across all functional modules. Each component enforces strict contracts, robust statistical methods, and exhaustive validation checks, ensuring reliability and academic credibility.

7. TESTING

7.1 Test Cases

Software testing is an essential verification phase that validates whether an application operates in strict compliance with its architectural requirements, statistical invariants, and error-handling contracts. The PARCS verification regime encompasses collector execution, ETL transformation, Parquet persistence, DuckDB query execution, robust statistical baselining, streak-based regression detection, temporal change correlation, FastAPI endpoint validation, and Streamlit dashboard rendering.

The test suite was executed against the project implementation using pytest. Table 7.1 details key formal test cases evaluated across all subsystems, reporting preconditions, expected behaviors, observed results, and pass/fail statuses.

Test ID	Feature Tested	Input / Precondition	Expected Result	Actual Result	Status
TC-01	SleepStudy Collection	powercfg /sleepstudy output	Extracts valid sessions with DRIPS % and drain rate	Sessions parsed successfully into CollectorResult	Pass
TC-02	Driver Snapshot Collection	Get-PnpDevice driver listing	Captures PnP device IDs, driver versions, providers	Structured driver records captured in CollectorResult	Pass
TC-03	Windows Update Collection	Get-HotFix patch query	Captures KB numbers, install dates, descriptions	Hotfix list extracted without error	Pass
TC-04	Kernel Event Log Query	Kernel-Power event log query	Filters Event IDs 506/507 within time window	Matching power transitions extracted cleanly	Pass
TC-05	Raw Data Storage	CollectorResult payload	Persists timestamped raw JSON in data/raw/	Raw JSON file created and referenced	Pass
TC-06	ETL Schema Validation	Raw session dictionaries	Normalizes timestamps and enforces canonical columns	DataFrame conforms to SLEEPSTUDY_COLUMNS	Pass
TC-07	Parquet Persistence	Normalized DataFrame	Writes columnar Parquet and reads back identically	Zero schema corruption or type drift	Pass
TC-08	DuckDB Direct SQL Query	Processed Parquet file	Executes SQL aggregations directly over Parquet	Sub-millisecond query execution without server	Pass

Test ID	Feature Tested	Input / Precondition	Expected Result	Actual Result	Status
TC-09	Baseline Construction	14 sessions (DEMO-MACHINE-READY)	Calculates Median, MAD, IQR; marks readiness='ready'	Median DRIPS=95.0, readiness='ready'	Pass
TC-10	Insufficient History Guard	4 sessions (DEMO-MACHINE-SPARSE)	Marks readiness='insufficient_history' without error	readiness='insufficient_history', no crash	Pass
TC-11	Zero-MAD Fallback	Sessions with identical values	Falls back to IQR/1.3490 or epsilon; flag=True	dispersion_scale > 0, zero_mad_fallback=True	Pass
TC-12	Point Anomaly Detection	Degraded session (DRIPS 55%)	Modified Z-score > 3.5; flags is_anomaly=True	Score=15.7, is_anomaly=True, status='anomaly'	Pass
TC-13	Sustained Regression Streak	4 consecutive degraded sessions	Requires min_consecutive=3; flags is_regression=True	Sustained regression flagged on sessions 3 and 4	Pass
TC-14	Streak Reset on Normal	Degraded then normal session	Streak resets immediately to 0; no false alarm	consecutive_count resets to 0	Pass
TC-15	Temporal Change Correlation	Regression + Driver update at -1.5h	Matches change in [-24h, +2h]; offset=-5400s	is_correlated=True, relation='before'	Pass
TC-16	Non-Causal Explanation	Correlated regression record	Explanation notes association; denies proof of causality	Explanation contains 'causation is not asserted'	Pass
TC-17	Scope Isolation (Subsequent)	12 reference + 4 eval sessions	Subsequent scope evaluates only 4 eval sessions	Evaluates source rows 12..15 against baseline	Pass
TC-18	Empty Subsequent Scope	reference_count = total sessions	Returns empty list without scoring reference sessions	Returns [] without falling back to self-scoring	Pass
TC-19	Scope Validation (HTTP 422)	evaluation_scope='invalid'	Rejects invalid scope; returns HTTP 422	HTTP 422 Unprocessable Entity returned	Pass
TC-20	Synthetic Data Provenance	DEMO-MACHINE-READY & SPARSE	Tracks is_synthetic: True per machine	machine_provenance accurately reflects demo status	Pass

Test ID	Feature Tested	Input / Precondition	Expected Result	Actual Result	Status
TC-21	Streamlit Dashboard AppTest	Render dashboard via AppTest	Renders 5 tabs and selectbox without exception	AppTest rendered cleanly; at.exception is None	Pass

7.2 Unit Testing

Unit testing isolates individual algorithms and functions, validating mathematical accuracy and edge-case handling before integration into higher-level workflows:

- **Statistical Baseline & Contamination Guard:** Tests in test_analytics_baseline.py verify that Median, MAD, and IQR match hand-calculated statistical values. The test_contamination_guard test proves that if a naive baseline is computed across both normal and degraded observations, the median shifts from 95.0 down to 72.5, whereas the partitioned reference baseline preserves the pure 95.0 median.
- **Streak Continuity & Directional Logic:** Tests in test_analytics_regression.py confirm that directional evaluation correctly treats lower DRIPS residency and higher battery drain rates as harmful. Chronological sort orders (start_time ascending) are verified against shuffled inputs. Intervening normal sessions or missing timestamps immediately reset the streak counter to zero.
- **Modified Z-Score Cutoff:** Tests in test_analytics_anomaly.py verify that Modified Z-scores accurately scale deviations by 1.4826 * MAD. Sessions with missing metrics or unready baselines produce explicit non-anomaly statuses without raising unhandled exceptions.
- **Temporal Windowing & Offset Calculation:** Tests in test_analytics_correlation.py verify that temporal offsets are computed accurately in seconds, classifying changes occurring before, during, or after sleep sessions according to configured time boundaries.

7.3 Integrated Testing

Integrated testing evaluates end-to-end interactions between collectors, ETL pipelines, Parquet/DuckDB storage, FastAPI routes, and the Streamlit dashboard client. All reported testing results represent findings from the formal acceptance audit executed on the project repository:

- **Automated Test Suite Result:** The automated test suite executed via pytest across 16 test modules achieved 253 passed tests, 0 failures, and 1 warning in 7.90 seconds.
- **Dashboard Client Integration:** All 3 client integration tests in test_dashboard_client.py passed, confirming that the client gracefully queries live endpoints and falls back to local in-process analytics when offline.
- **Live API Verification:** Sixteen live HTTP API endpoint tests in test_api.py passed, verifying /health, /api/telemetry/summary, /api/analytics/baselines, /api/analytics/anomalies, /api/analytics/regressions, /api/analytics/correlations, /api/changes, and /api/demo/seed.
- **Streamlit UI Rendering:** The Streamlit AppTest framework executed src/dashboard/app.py without exceptions, verifying that all 5 tabs and machine selector dropdowns render cleanly.

Maintenance Deprecation Warnings: During test execution, two non-blocking deprecation warnings were recorded: (1) A StarletteDeprecationWarning from FastAPI TestClient noting that using httpx with starlette.testclient is deprecated and recommending httpx2; and (2) A Streamlit deprecation warning regarding the use_container_width parameter in st.dataframe and st.button. Both warnings are

documented as non-blocking maintenance backlog items and were deliberately preserved without unnecessary code churn.

8. RESULTS AND CONCLUSION

8.1 Results

The implementation and testing of PARCS demonstrates the feasibility and effectiveness of a data-engineering-first architecture for diagnosing Windows Modern Standby regressions. The primary functional and analytical outcomes of the MVP release include:

- **1. End-to-End Pipeline Execution:** The multi-stage pipeline successfully bridges Windows telemetry collection, raw JSON preservation, schema validation, columnar Parquet persistence, and embedded DuckDB SQL execution without external database infrastructure.
- **2. Baseline Fidelity:** The baseline engine establishes robust Median, MAD, and IQR metrics strictly isolated by machine identity. On DEMO-MACHINE-READY, 14 historical baseline sessions established a clean reference median of 95.0% DRIPS residency and 2,400 mW battery drain rate.
- **3. Sustained Regression Isolation:** The regression detector accurately identified 4 consecutive degraded sessions (DRIPS ~ 55%, battery drain ~ 5,400 mW), triggering sustained regression alerts on sessions 3 and 4 while rejecting single-session transient anomalies.
- **4. Actionable Change Correlation:** The correlation engine successfully matched the regression onset with an Intel Wi-Fi 6 AX201 driver update occurring 1.5 hours prior (temporal_offset = -5,400 seconds), providing clear investigative evidence.
- **5. False Alarm Suppression:** On DEMO-MACHINE-SPARSE (4 sessions), the system correctly assigned readiness = 'insufficient_history' and status = 'insufficient_history', completely preventing false regression alarms.

8.2 Limitations

To maintain rigorous academic integrity, several important operational limitations must be explicitly acknowledged:

- **Synthetic Demonstration Data:** All multi-session regression and correlation results reported in this MVP audit are derived from deterministic synthetic fixtures (DEMO-MACHINE-READY and DEMO-MACHINE-SPARSE). While the SleepStudy, driver, update, and event log collectors are fully implemented to execute real Windows commands, multi-week physical laptop telemetry collection was not deployed or validated end-to-end.
- **Temporal Association vs Causation:** The correlation engine identifies temporal association within configured time windows ([-24h, +2h]). It does NOT prove that a recorded driver update or Windows patch caused the regression. Unrecorded background factors may be responsible.
- **Manual Invocation Model:** PARCS currently operates as a local desktop application invoked via CLI, API, or Streamlit. It does not run as a continuous Windows background service or Windows Task Scheduler daemon.

8.3 Future Enhancements

Future research and engineering extensions can significantly expand the capabilities of PARCS beyond the MVP release:

- **1. Automated Background Daemon:** Implement a native Windows Service daemon or Task Scheduler trigger that automatically runs SleepStudy collection and driver snapshots immediately upon resume from Modern Standby (Event ID 507).

- **2. Causal Discovery Modeling:** Incorporate causal discovery algorithms (such as the Peter-Clark algorithm or Granger causality) to distinguish spurious temporal correlations from genuine causal relationships.
- **3. Enterprise Fleet Aggregation:** Extend the local Parquet data lake to support multi-node fleet synchronization via Apache Arrow Flight or object storage, enabling enterprise IT administrators to correlate regressions across thousands of enterprise laptops.
- **4. Automated Rollback Guidance:** Introduce automated rollback recommendations for recently updated drivers when sustained regressions are strongly correlated.

8.4 Conclusion

PARCS demonstrates that client-side Windows Modern Standby power regressions can be systematically diagnosed through an automated, data-engineering-first architecture. By combining modular telemetry collection, immutable Parquet persistence, embedded DuckDB querying, robust non-parametric statistics (Median, MAD, IQR), streak-based regression detection, and explainable change correlation, PARCS bridges the longstanding divide between stateless low-level diagnostics and actionable engineering insights. The verified MVP release establishes a rigorous, privacy-preserving, and mathematically grounded foundation for next-generation PC power management analytics.

9. APPENDICES

9.1 Demonstration and Startup Instructions

PARCS provides a unified Command Line Interface (CLI) via main.py to seed synthetic demonstration datasets, launch the FastAPI backend, or start the Streamlit visual dashboard:

Step 1: 1. Virtual Environment Setup & Dependency Installation:

```
# Navigate to project root
cd c:\Users\shant\OneDrive\Documents\PROJECTS\PARCS

# Create and activate virtual environment
python -m venv .venv
.venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt
```

Step 2: 2. Seed Deterministic Synthetic Demonstration Datasets:

```
# Seeds 22 sessions and 4 changes into Parquet storage
python main.py seed-demo

# Output:
# Seeded demo datasets successfully: {'success': True, 'session_records': 22,
'change_records': 4}
```

Step 3: 3. Launch FastAPI REST Backend Service:

```
# Start API on localhost:8000
python main.py api --host 127.0.0.1 --port 8000

# Interactive Swagger UI available at http://127.0.0.1:8000/docs
```

Step 4: 4. Launch Streamlit Visual Analytics Dashboard:

```
# Start Streamlit dashboard in a new terminal
python main.py dashboard

# Automatically opens browser at http://localhost:8501
```

Step 5: 5. Execute Complete Automated Test Suite:

```
# Run full pytest test suite
.venv\Scripts\pytest

# Result: 253 passed, 1 warning in 7.90s
```

9.2 Selected API Examples

The following JSON payloads represent actual responses returned by the FastAPI backend during verification:

Endpoint 1: GET /api/telemetry/summary:


```
{
  "total_sessions": 22,
  "total_changes": 4,
  "machines": ["DEMO-MACHINE-READY", "DEMO-MACHINE-SPARSE"],
  "date_range": {
    "start": "2026-08-10T10:00:00Z",
    "end": "2026-09-04T14:30:00Z"
  },
  "datasets_available": {
    "sleepstudy": true,
    "changes": true
  },
  "is_synthetic": true,
  "machine_provenance": {
    "DEMO-MACHINE-READY": true,
    "DEMO-MACHINE-SPARSE": true
  },
  "status": "ok"
}
```


Endpoint 2: GET /api/analytics/regressions?machine_id=DEMO-MACHINE-READY:


```
[
  {
    "machine_id": "DEMO-MACHINE-READY",
    "start_time": "2026-09-01T11:30:00Z",
    "end_time": "2026-09-01T12:00:00Z",
    "metric": "drips_percent",
    "observed_value": 55.0,
    "baseline_median": 95.0,
    "deviation_score": 15.68,
    "direction": "lower",
    "is_regression": true,
    "status": "regression",
    "consecutive_count": 3,
    "required_consecutive": 3,
    "reason": "Sustained degradation of drips_percent (observed 55.0% vs baseline 95.0%)
across 3 sessions."
  }
]
```


Endpoint 3: GET /api/analytics/correlations?machine_id=DEMO-MACHINE-READY:


```
[
  {
    "machine_id": "DEMO-MACHINE-READY",
    "metric": "drips_percent",
    "change_id": "PCI\\VEN_8086&DEV_06F0",
    "change_type": "driver",
    "change_description": "Intel(R) Wi-Fi 6 AX201 160MHz v22.130.0.5",
    "temporal_offset_seconds": -5400.0,
    "temporal_relation": "before",
    "is_correlated": true,
    "reason": "Driver change observed 1.5h before regression onset (temporal association
only, causation is not asserted)."
  }
]
```

9.3 Representative Source Code

The following representative code excerpts illustrate core algorithmic implementations from the PARCS repository:

Listing 9.1: Excerpt from src/analytics/baseline.py (Robust Scaling & Zero-MAD Fallback):

```
def _calculate_metric_baseline(series: pd.Series, min_obs: int) -> dict[str, object]:
    valid_vals = series.dropna().to_numpy(dtype=float)
    if len(valid_vals) < min_obs:
        return {'readiness': 'insufficient_history', 'median': None, 'mad': None}

    med = float(np.median(valid_vals))
    abs_devs = np.abs(valid_vals - med)
    mad = float(np.median(abs_devs))
    iqr = float(np.percentile(valid_vals, 75) - np.percentile(valid_vals, 25))

    # Documented zero-MAD fallback strategy
    if mad > 0.0:
        scale = mad * 1.4826
        fallback = False
    elif iqr > 0.0:
        scale = iqr / 1.3490
        fallback = True
    else:
        scale = 1e-4  # Epsilon positive scale
        fallback = True

    return {'readiness': 'ready', 'median': med, 'mad': mad, 'iqr': iqr,
            'dispersion_scale': scale, 'zero_mad_fallback': fallback}
```

Listing 9.2: Excerpt from src/api/service.py (Prospective Reference Partitioning):

```
@staticmethod
def partition_machine_features(df: pd.DataFrame, min_obs: int = 10, ref_count: int = None):
    # Deterministic chronological sort: start_time asc, end_time asc
    df_sorted = df.sort_values(by=['start_time', 'end_time'], na_position='last')
    n_rows = len(df_sorted)

    if ref_count is not None:
        k = max(0, min(ref_count, n_rows))
        return df_sorted.iloc[:k].copy(), df_sorted.iloc[k:].copy()

    # Default policy for sparse history (<10 sessions)
    if n_rows <= min_obs:
        return df_sorted.copy(), df_sorted.copy()

    # Chronological 75% reference baseline split
    k = max(min_obs, int(n_rows * 0.75))
    return df_sorted.iloc[:k].copy(), df_sorted.iloc[k:].copy()
```

9.4 User Documentation

PARCS is designed for seamless operation by systems engineers, hardware validation teams, and IT administrators. The application provides interactive telemetry exploration via the Streamlit dashboard:

- **Machine Selection:** Use the top dropdown to choose between monitored machines. A warning badge is displayed if the machine uses synthetic demonstration data.

- **Parameter Calibration:** Adjust the 'Harmful Deviation Threshold' slider (default 2.0) and 'Required Consecutive Sessions' (default 3) in the sidebar to calibrate detection sensitivity.
- **Tabular Exploration:** Navigate between the 5 tabs to inspect statistical baselines, view flagged anomalies, review regression timelines, and examine correlated driver updates or Windows patches.

9.5 Glossary

The following glossary defines critical technical terms utilized throughout the PARCS project report:

Technical Term	Formal Definition
Modern Standby (S0ix)	A low-power hardware/software platform architecture where the system remains in ACPI S0 while the SoC enters quiescent low-power idle states.
DRIPS	Deepest Runtime Idle Power State; the lowest power hardware sleep state achievable by modern processors and SoCs.
Median Absolute Deviation (MAD)	A robust measure of statistical dispersion defined as the median of the absolute deviations from the dataset median.
Modified Z-Score	A robust outlier detection metric utilizing median and MAD to score deviations without sensitivity to extreme sample outliers.
Sustained Regression	A persistent performance degradation requiring a minimum run of consecutive qualifying degraded sessions in chronological order.
Temporal Correlation	An observed statistical association between an event and a regression onset based on temporal proximity, explicitly distinguished from causal proof.
Apache Parquet	An open-source, column-oriented data storage format providing high-performance compression and encoding schemes.
DuckDB	An embedded, in-process vectorized SQL analytical database engine optimized for executing fast queries over local Parquet files.
FastAPI	A modern, high-performance web framework for building REST APIs with Python 3.8+ based on standard OpenAPI and Pydantic validation.
Streamlit	An open-source Python framework for rapidly creating interactive data-driven web applications and dashboards.
Evaluation Scope	The filtering boundary for analytics evaluation: 'subsequent' evaluates post-reference sessions only; 'all' scores the entire dataset.
Data Provenance	The documentation of the origin and processing history of telemetry data, explicitly tracking real versus synthetic records.

10. BIBLIOGRAPHY

The bibliography lists authoritative academic publications, peer-reviewed textbooks, and official technical documentation consulted during the architecture, design, and verification of PARCS. References are organized into Books & Research Publications and Official Technical Documentation.

10.1 Books and Research Publications

1. Abadi, D. J., Madden, S. R., and Hachem, N. (2008). "Column-Stores vs. Row-Stores: How Different Are They Really?" Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data, pp. 967–980. https://doi.org/10.1145/1376616.1376712

2. Basseville, M., and Nikiforov, I. V. (1993). Detection of Abrupt Changes: Theory and Application. Prentice Hall, Englewood Cliffs, NJ.

3. Hampel, F. R. (1974). "The Influence Curve and Its Role in Robust Estimation." Journal of the American Statistical Association, 69(346), pp. 383–393. https://doi.org/10.1080/01621459.1974.10482962

4. Iglewicz, B., and Hoaglin, D. C. (1993). How to Detect and Handle Outliers. ASQC Basic References in Quality Control, Vol. 16, Milwaukee, WI: ASQC Quality Press.

5. Oliner, A., Ganapathi, A., and Xu, W. (2012). "Advances and Challenges in Log Analysis." Communications of the ACM, 55(2), pp. 55–61. https://doi.org/10.1145/2076450.2076466

6. Pearl, J. (2009). Causality: Models, Reasoning, and Inference (2nd ed.). Cambridge University Press, Cambridge, UK.

7. Popp, M., Ecker, D., and Brand, O. (2018). "Power Management Architecture for Low-Power System-on-Chips." IEEE Transactions on Circuits and Systems I: Regular Papers, 65(9), pp. 2910–2921.

8. Raasveldt, M., and Mühleisen, H. (2019). "DuckDB: An Embeddable Analytical Database." Proceedings of the 2019 ACM SIGMOD International Conference on Management of Data, pp. 1981–1984. https://doi.org/10.1145/3299869.3320212

9. Rousseeuw, P. J., and Croux, C. (1993). "Alternatives to the Median Absolute Deviation." Journal of the American Statistical Association, 88(424), pp. 1273–1283. https://doi.org/10.1080/01621459.1993.10476408

10.2 Official Technical Documentation

10. ACPI Specification Working Group. (2022). Advanced Configuration and Power Interface (ACPI) Specification, Version 6.5. Unified Extensible Firmware Interface (UEFI) Forum. https://uefi.org/specifications

11. Apache Software Foundation. (2023). Apache Parquet Documentation. https://parquet.apache.org/documentation/

12. DuckDB Foundation. (2023). DuckDB Documentation: In-Process Analytical SQL Engine. https://duckdb.org/docs/

13. Microsoft Corporation. (2023). Modern Standby Architecture and Diagnostics (powercfg /sleepstudy). Microsoft Hardware Developer Center. https://learn.microsoft.com/en-us/windows-hardware/design/device-experiences/modern-standby

14. Microsoft Corporation. (2023). Windows Event Log Reference (Microsoft-Windows-Kernel-Power). Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/wes/windows-event-log

15. Python Software Foundation. (2024). Python 3 Documentation. https://docs.python.org/3/

16. Streamlit Inc. (2024). Streamlit Documentation: Modern Data Applications. https://docs.streamlit.io/

17. Tiangolo, S. (2024). FastAPI Documentation: Modern Python Web Framework. https://fastapi.tiangolo.com/